

Pagination in Node.js and MongoDB

This tutorial explains how to manually implement **pagination** in a Node.js + Express + MongoDB API, and how to consume that paginated API on the frontend using Twig and JavaScript.

The entire pagination logic is implemented **without using mongoose-paginate or any other package**, giving full control over skip/limit queries and pagination UI.

1. Understanding Pagination

1.1 What is Pagination?

Pagination divides large datasets into smaller parts (pages). Example: You want to show **3 records per page**.

Page 1 → Records 1–3 Page 2 → Records 4–6 Page 3 → Records 7–9 and so on.

2. Required Calculations

2.1 Total Number of Pages

Formula:

```
totalPages = Math.ceil(totalRecords / limit);
```

Example:

- totalRecords = 7
- limit = 3
- $7 / 3 = 2.33 \rightarrow \text{ceil} \rightarrow \mathbf{3 \text{ pages}}$

2.2 Skip Formula

To fetch correct records for each page:

```
skip = (page - 1) * limit;
```

Example:

Page	Calculation	Skip
1	$(1-1) \times 3 = 0$	0

Page	Calculation	Skip
2	$(2-1) \times 3 = 3$	3
3	$(3-1) \times 3 = 6$	6

These formulas are implemented in the backend API.

3. Building the Pagination API in Node.js

3.1 Backend Route Setup (Express + Mongoose)

Step 1: Extract page and limit from query string

```
let page = parseInt(req.query.page) || 1;
let limit = parseInt(req.query.limit) || 5;
```

Step 2: Calculate skip

```
let skip = (page - 1) * limit;
```

Step 3: Count total records

```
let total = await Student.countDocuments({});
```

Step 4: Fetch paginated records

```
let students = await Student.find({})
  .skip(skip)
  .limit(limit);
```

Step 5: Return response JSON

```
res.json({
  total: total,
  page: page,
  limit: limit,
  totalPages: Math.ceil(total / limit),
  students: students
});
```

4. Testing the API in Postman

Example URL:

```
GET http://localhost:3000/api/students?page=2&limit=5
```

Response structure:

```
{
  "total": 8,
  "page": 2,
  "limit": 5,
  "totalPages": 2,
  "students": [ ... ]
}
```

5. Frontend Implementation (HTML + Twig + JS)

5.1 Twig Template Structure

```
<table>
  <thead>
    <tr>
      <th>ID</th>
      <th>Name</th>
      <th>Email</th>
    </tr>
  </thead>

  <tbody id="studentTableBody">
    {# rows will be injected using JS fetch() #}
  </tbody>
</table>

<ul id="pagination" class="pagination justify-content-center"></ul>
```

6. Fetching Paginated Data from Frontend

```
let currentPage = 1;
let currentSearch = '';

async function fetchStudents(search = '', page = 1) {
  currentPage = page;
  currentSearch = search;

  let apiUrl = `/api/students?search=${search}&page=${page}&limit=3`;

  let response = await fetch(apiUrl);
  let data = await response.json();

  renderTable(data.students);
  renderPagination(data.totalPages);
}
```

7. Rendering Table Rows in the Frontend

```
function renderTable(students) {
  const tbody = document.getElementById("studentTableBody");
  tbody.innerHTML = "";

  students.forEach((student) => {
    tbody.innerHTML += `
      <tr>
        <td>${student._id}</td>
        <td>${student.name}</td>
        <td>${student.email}</td>
      </tr>
    `;
  });
}
```

8. Rendering Pagination Links

8.1 Main pagination rendering function

```
function renderPagination(totalPages) {
  const container = document.getElementById('pagination');
  container.innerHTML = '';

  // Previous Button
  let prev = document.createElement('li');
```

```
prev.className = 'page-item ' + (currentPage === 1 ? 'disabled' : '');
prev.innerHTML = `Previous</a>`;
prev.addEventListener\('click', \(e\) => {
  e.preventDefault\(\);
  if \(currentPage > 1\) fetchStudents\(currentSearch, currentPage - 1\);
}\);
container.appendChild\(prev\);

// Page Numbers
for \(let i = 1; i <= totalPages; i++\) {
  let li = document.createElement\('li'\);
  li.className = 'page-item ' + \(currentPage === i ? 'active' : ''\);
  li.innerHTML = `\\${i}</a>`;

  li.addEventListener\\('click', \\(e\\) => {
    e.preventDefault\\(\\);
    fetchStudents\\(currentSearch, i\\);
  }\\);

  container.appendChild\\(li\\);
}

// Next Button
let next = document.createElement\\('li'\\);
next.className = 'page-item ' + \\(currentPage === totalPages ? 'disabled' :
''\\);
next.innerHTML = `Next</a>`;
next.addEventListener\\\('click', \\\(e\\\) => {
  e.preventDefault\\\(\\\);
  if \\\(currentPage < totalPages\\\) fetchStudents\\\(currentSearch, currentPage +
1\\\);
}\\\);
container.appendChild\\\(next\\\);
}
```

9. Integrating Pagination with Search

When applying search, the pagination must reset to the first page.

```
function searchStudents() {
  const input = document.getElementById('searchInput').value;
  fetchStudents(input, 1);
}
```

10. Summary of What You Implemented

- Manual pagination using **page**, **limit**, and **skip**
 - Backend calculations for:
 - `totalRecords`
 - `totalPages`
 - `skip`
 - `records page-wise`
 - Frontend pagination UI (Previous, 1, 2, 3, Next)
 - Active page highlighting
 - Integration with search
 - Twig templates used instead of EJS
 - Complete API + UI linkage
-